

```
Building and testing Config-Tiny-2.20 ... OK
Successfully installed Config-Tiny-2.20
1 distribution installed
```

We then move in to the directory that has the 64-bit Perl version 5.16 installed, run the *portablesell.bat* command and install the same module:

```
C:\my64bitperl516>cpanm Config::Tiny
--> Working on Config::Tiny
Fetching http://www.cpan.org/authors/id/R/RS/RSVAGE/Config-
Tiny-2.20.tgz ... OK
Configuring Config-Tiny-2.20 ... OK
Building and testing Config-Tiny-2.20 ... OK
Successfully installed Config-Tiny-2.20
1 distribution installed
```

And in the 64-bit Perl version 5.18 directory as well, don't forget to run *portablesell.bat*:

```
C:\my64bitperl518>cpanm Config::Tiny
--> Working on Config::Tiny
Fetching http://www.cpan.org/authors/id/R/RS/RSVAGE/Config-
Tiny-2.20.tgz ... OK
Configuring Config-Tiny-2.20 ... OK
Building and testing Config-Tiny-2.20 ... OK
Successfully installed Config-Tiny-2.20
1 distribution installed
```

Now, let's write a script and run it against each Perl version.



Note: When you run scripts, you don't need to change to the specific directory where the desired Perl Version is installed. Just run the *portablesell.bat* from the desired directory. If I run `C:\my32bitperl516\portablesell.bat`, I get to use the 32 bit Perl Version 5.16, and after I am done and want to use Perl Version 5.18 64 bit, I just run `C:\my64bitperl518\portablesell.bat`. You just need to hit *exit* a few times to get back to the earlier version of Perl.

You just need to hit *Exit* a few times to get back to the earlier version of Perl.

Let's write a small script that uses the *Smartmatch* feature. Nothing fancy, but let's create two arrays, each with names of different Linux flavours, and then use the *Smartmatch* operator `~` to compare them. It's on the way to being deprecated and has moved to 'experimental' in Perl Version 5.18.

```
#!/usr/bin/perl
#script name pl1.pl
use warnings;
use strict;
my @firstarray = qw (debian redhat ubuntu centos mint stella
elementaryos);
```

```
my @secondarray = qw (debian redhat ubuntu centos mint stella
elementaryos);
print "The arrays have the same elements!\n" if @firstarray
~~ @secondarray;
```

When it's run using Perl version 5.16 (whether 32- or 64-bit will not matter here), you will get the following output:

```
C:\Users\pmu>perl --version | find /I "version"
This is perl 5, version 16, subversion 3 (v5.16.3) built for
MSWin32-x86-multi-thread
C:\Users\pmu>perl pl1.pl
The arrays have the same elements!!
```

And with Perl version 5.18, you'll get the following output:

```
C:\Users\pmu>perl --version | find /I "version"
This is perl 5, version 18, subversion 2 (v5.18.2) built for
MSWin32-x64-multi-thread
C:\Users\pmu>perl pl1.pl
Smartmatch is experimental at pl1.pl line 8.
The arrays have the same elements!!
```

Note the line in the output that says 'Smartmatch is experimental at pl1.pl line 8'. As mentioned before, *Smartmatch* has been moved to 'experimental' with Perl version 5.18, and that's why we get the message when the script is run with Perl version 5.18. If you were to run a script with 'given-when' statements, you'd get something similar. This way, you can run the same script against different Perl versions and see how it works.

You can also copy the required folder to a USB drive, paste it into another computer, run the *portablesell.bat* command, and you've got Perl running in there too!

Contrary to what you might have heard from fellow techies or read on the Internet, Perl is very much alive and is still being actively used. A number of organisations, ranging from financial giants, media companies and many others, still use Perl for a lot of critical tasks. It's truly disheartening to see that such a powerful language is sometimes mistakenly referred to as 'write only', 'ugly', etc. It's not a shortcoming of a language if people write unreadable/unmaintainable code in it. One can write well written/formatted code in any language, including Perl. This is one language that is very versatile and is bound to meet your needs.

Yes, there is a steep learning curve with the advanced topics, but once you start getting the hang of it, you'll use Perl a lot more than you'd imagine. 

By: Pritesh Manohar Ugrankar

The author is an open source enthusiast who loves Perl and Linux. You can reach him at pritesh.ugrankar@gmail.com